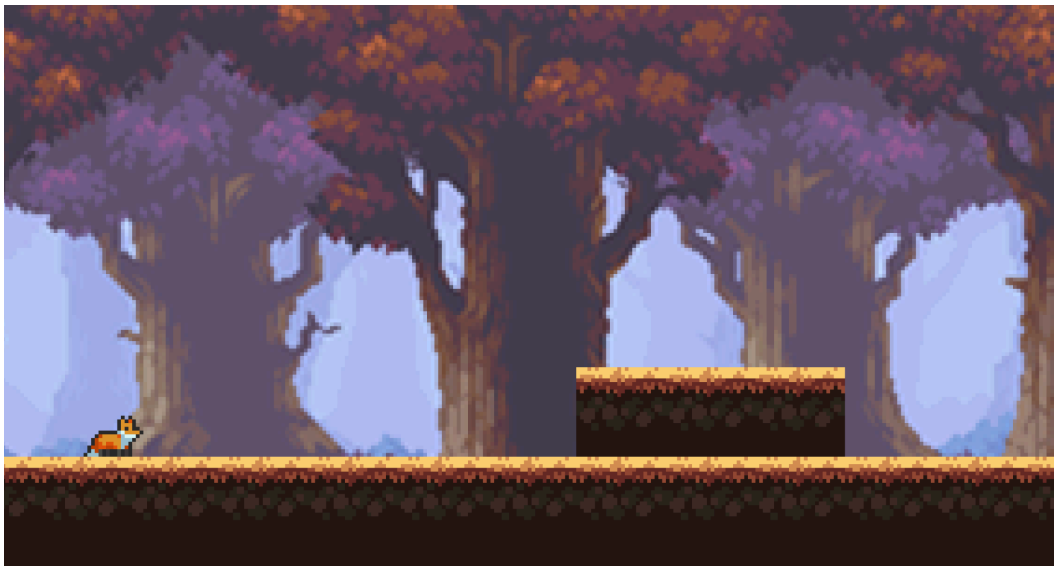
Process & Decision Documentation

Project/Assignment Decisions

- Added a sound effect when the user makes a jump
- Added background music to match the ambiance of the environment, triggered by the click of a mouse

Role-Based Process Evidence



- Sound works after trial and error

Entry Header

Name: Jenny Dang

Primary responsibility for this work: Add sound/effects to the code

Goal of Work Session

- To add sound/effects

Tools, Resources, or Inputs Used

- GenAI tools (ChatGPT 5.2)
- Code by Dr. Karen Cochrane and David Han

GenAI Documentation

Date Used: March 9, 2026

Tool Disclosure: ChatGPT 5.2

Purpose of Use: Help edit code and comments to add sound to the game

Summary of Interaction: The tool helped me to add sound files to the game, making it more entertaining

Human Decision Point(s): I overruled when the sound was not working properly and the sound effect was not working when the user made a jump. I fixed this by going over the code and finding the errors.

Integrity & Verification Note: I checked for accuracy by testing the game in a live server.

Scope of GenAI Use: GenAI did not contribute to my prompts and my decision-making.

Limitations or Misfires: The tool misunderstood where to place the sound effect when the user jumps/

Summary of Process (Human + Tool)

- Testing and revising
- Prompting ChatGPT
- Picking the right sound
- Reiterating the GenAI to get the correct outcome

Decision Points & Trade-offs

- I considered which sound effects were suitable for the game

Verification & Judgement

- Playtesting on a live server

Limitations, Dead Ends, or Open Questions

- At first, the sound was not working because I didn't add the correct code in the index
- Next time, I could make a button where you can mute the background music

Appendix

ME: add sound to this code specifically when the fox jumps, it makes a noise /* Week 6 — Example 2: Tile-Based Level & Basic Movement Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han Date: Feb. 26, 2026 Controls: A or D (Left / Right Arrow) Horizontal movement W (Up Arrow) Jump Space Bar Attack Tile key: g = groundTile.png (surface ground) d = groundTileDeep.png (deep ground, below surface)

= empty (no sprite) */ let player; let playerImg, bgImg; let playerAnis = { idle: { row: 0, frames: 4, frameDelay: 10 }, run: { row: 1, frames: 4, frameDelay: 3 }, jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 }, attack: { row: 3, frames: 6, frameDelay: 2 }, }; let ground, groundDeep; let groundImg, groundDeepImg; let attacking = false; // track if the player is attacking let attackFrameCounter = 0; // tracking attack animation // --- TILE MAP --- // an array that uses the tile key to create the level let level = [" ", " ", " ", " ", " ", " ggg ", "gggggggggggggg", // surface ground "ddddddddddddddd", // deep ground]; // --- LEVEL CONSTANTS --- // camera view size const VIEWW = 320, VIEWH = 180; // tile width & height const TILE_W = 24, TILE_H = 24; // size of individual animation frames const FRAME_W = 32, FRAME_H = 32; // Y-coordinate of player start (4 tiles above the bottom) const MAP_START_Y = VIEWH - TILE_H * 4; // gravity const GRAVITY = 10; function preload() { // --- IMAGES --- playerImg = loadImage("assets/foxSpriteSheet.png"); bgImg = loadImage("assets/combinedBackground.png"); groundImg = loadImage("assets/groundTile.png"); groundDeepImg = loadImage("assets/groundTileDeep.png"); } function setup() { // pixelated rendering with autoscaling new Canvas(VIEWW, VIEWH, "pixelated"); // needed to correct an visual artifacts from attempted antialiasing allSprites.pixelPerfect = true; world.gravity.y = GRAVITY; // --- TILE GROUPS --- ground = new Group(); ground.physics = "static"; ground.img = groundImg; ground.tile = "g"; groundDeep = new Group(); groundDeep.physics = "static"; groundDeep.img = groundDeepImg; groundDeep.tile = "d"; // a Tiles object creates a level based on the level map array (defined at the beginning) new Tiles(level, 0, 0, TILE_W, TILE_H); // --- PLAYER --- player = new Sprite(FRAME_W, MAP_START_Y, FRAME_W, FRAME_H); // create the player player.spriteSheet = playerImg; // use the sprite sheet player.rotationLock = true; // turn off rotations (player shouldn't rotate) // player animation parameters player.anis.w = FRAME_W; player.anis.h = FRAME_H; player.anis.offset.y = -4; // offset the collision box up player.addAnis(playerAnis); // add the player animations defined earlier player.ani = "idle"; // default to the idle animation player.w = 18; // set the width of the collsion box player.h = 20; // set the height of the collsion box player.friction = 0; // set the friciton to 0 so we don't stick to walls player.bounciness = 0; // set the bounciness to 0 so the player doesn't bounce // --- GROUND SENSOR --- for use when detecting if the player is standing on the ground sensor = new Sprite(); sensor.x = player.x; sensor.y = player.y + player.h / 2; sensor.w = player.w; sensor.h = 2; sensor.mass = 0.01; sensor.removeColliders(); sensor.visible = false; let sensorJoint = new GlueJoint(player, sensor); sensorJoint.visible = false; } function draw() { // --- BACKGROUND --- camera.off(); imageMode(CORNER); image(bgImg, 0, 0, bgImg.width, bgImg.height); camera.on(); // --- PLAYER CONTROLS --- // first check to see if the player is on the

ground let grounded = sensor.overlapping(ground); // -- ATTACK INPUT -- if (grounded && !attacking && kb.presses("space")) { attacking = true; attackFrameCounter = 0; player.vel.x = 0; player.ani.frame = 0; player.ani = "attack"; player.ani.play(); // plays once to end } // -- JUMP -- if (grounded && kb.presses("up")) { player.vel.y = -4; } // --- STATE MACHINE --- if (attacking) { attackFrameCounter++; // Attack lasts ~6 frames * frameDelay 2 = 12 cycles (adjust if needed) if (attackFrameCounter > 12) { attacking = false; attackFrameCounter = 0; } } else if (!grounded) { player.ani = "jump"; player.ani.frame = player.vel.y < 0 ? 0 : 1; } else { player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle"; } // --- MOVEMENT --- if (!attacking) { player.vel.x = 0; if (kb.pressing("left")) { player.vel.x = -1.5; player.mirror.x = true; } else if (kb.pressing("right")) { player.vel.x = 1.5; player.mirror.x = false; } } // --- KEEP IN VIEW --- player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2); }

AI:

```
/*
  Week 6 — Example 2: Tile-Based Level & Basic Movement


  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026


  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack


  Tile key:
    g = groundTile.png       (surface ground)
    d = groundTileDeep.png   (deep ground, below surface)
      = empty (no sprite)
*/
```

```
let player;
let playerImg, bgImg;
let jumpSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep;
let groundImg, groundDeepImg;

let attacking = false;
let attackFrameCounter = 0;

let level = [
  "              ",
  "              ",
  "              ",
  "              ",
  "              ",
  "       ggg    ",
  "ggggggggggggggg",
  "dddddddddddddd",
```

```
];

const VIEWW = 320,
  VIEWH = 180;

const TILE_W = 24,
  TILE_H = 24;

const FRAME_W = 32,
  FRAME_H = 32;

const MAP_START_Y = VIEWH - TILE_H * 4;

const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgImg = loadImage("assets/combinedBackground.png");
  groundImg = loadImage("assets/groundTile.png");
  groundDeepImg = loadImage("assets/groundTileDeep.png");

  jumpSound = loadSound("assets/foxJump.wav");
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
```

```
allSprites.pixelPerfect = true;

world.gravity.y = GRAVITY;

ground = new Group();
ground.physics = "static";
ground.img = groundImg;
ground.tile = "g";

groundDeep = new Group();
groundDeep.physics = "static";
groundDeep.img = groundDeepImg;
groundDeep.tile = "d";

new Tiles(level, 0, 0, TILE_W, TILE_H);

player = new Sprite(FRAME_W, MAP_START_Y, FRAME_W, FRAME_H);
player.spriteSheet = playerImg;
player.rotationLock = true;

player.anis.w = FRAME_W;
player.anis.h = FRAME_H;
player.anis.offset.y = -4;
player.addAnis(playerAnis);
player.ani = "idle";
```

```
  player.w = 18;
  player.h = 20;
  player.friction = 0;
  player.bounciness = 0;

  sensor = new Sprite();
  sensor.x = player.x;
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;

  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;
}

function draw() {
  camera.off();
  imageMode(CORNER);
  image(bgImg, 0, 0, bgImg.width, bgImg.height);
  camera.on();

  let grounded = sensor.overlapping(ground);
```

```
if (grounded && !attacking && kb.presses("space")) {
  attacking = true;
  attackFrameCounter = 0;
  player.vel.x = 0;
  player.ani.frame = 0;
  player.ani = "attack";
  player.ani.play();
}

if (grounded && kb.presses("up")) {
  player.vel.y = -4;
  jumpSound.play();
}

if (attacking) {
  attackFrameCounter++;
  if (attackFrameCounter > 12) {
    attacking = false;
    attackFrameCounter = 0;
  }
} else if (!grounded) {
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}
```

```
  if (!attacking) {
    player.vel.x = 0;

    if (kb.pressing("left")) {
      player.vel.x = -1.5;
      player.mirror.x = true;
    }
    else if (kb.pressing("right")) {
      player.vel.x = 1.5;
      player.mirror.x = false;
    }
  }

  player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2);
}
```

ME:

okay my sound is named foxjump.mp3 can you give me the full code

AI:

```
/*
  Week 6 — Tile-Based Level & Movement
  Added jump sound: foxjump.mp3
*/

let player;
```

```
let sensor;

let playerImg, bgImg;
let groundImg, groundDeepImg;
let jumpSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep;

let attacking = false;
let attackFrameCounter = 0;

let level = [
  "              ",
  "              ",
  "              ",
  "              ",
  "              ",
  "       ggg    ",
  "ggggggggggggggg",
```

```
  "ddddddddddddd",
];

const VIEWW = 320;
const VIEWH = 180;

const TILE_W = 24;
const TILE_H = 24;

const FRAME_W = 32;
const FRAME_H = 32;

const MAP_START_Y = VIEWH - TILE_H * 4;

const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgImg = loadImage("assets/combinedBackground.png");
  groundImg = loadImage("assets/groundTile.png");
  groundDeepImg = loadImage("assets/groundTileDeep.png");

  jumpSound = loadSound("assets/foxjump.mp3");
}

function setup() {
```

```
new Canvas(VIEWW, VIEWH, "pixelated");

allSprites.pixelPerfect = true;

world.gravity.y = GRAVITY;

ground = new Group();
ground.physics = "static";
ground.img = groundImg;
ground.tile = "g";

groundDeep = new Group();
groundDeep.physics = "static";
groundDeep.img = groundDeepImg;
groundDeep.tile = "d";

new Tiles(level, 0, 0, TILE_W, TILE_H);

player = new Sprite(FRAME_W, MAP_START_Y, FRAME_W, FRAME_H);
player.spriteSheet = playerImg;
player.rotationLock = true;

player.anis.w = FRAME_W;
player.anis.h = FRAME_H;
player.anis.offset.y = -4;
player.addAnis(playerAnis);
```

```
  player.ani = "idle";

  player.w = 18;
  player.h = 20;
  player.friction = 0;
  player.bounciness = 0;

  sensor = new Sprite();
  sensor.x = player.x;
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;

  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;
}

function draw() {
  camera.off();
  imageMode(CORNER);
  image(bgImg, 0, 0, bgImg.width, bgImg.height);
  camera.on();
```

```
let grounded = sensor.overlapping(ground);

if (grounded && !attacking && kb.presses("space")) {
  attacking = true;
  attackFrameCounter = 0;
  player.vel.x = 0;
  player.ani.frame = 0;
  player.ani = "attack";
  player.ani.play();
}

if (grounded && kb.presses("up")) {
  player.vel.y = -4;

  if (jumpSound && !jumpSound.isPlaying()) {
    jumpSound.play();
  }
}

if (attacking) {
  attackFrameCounter++;
  if (attackFrameCounter > 12) {
    attacking = false;
    attackFrameCounter = 0;
  }
}
```

```
  else if (!grounded) {
    player.ani = "jump";
    player.ani.frame = player.vel.y < 0 ? 0 : 1;
  }
  else {
    player.ani =
      kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
  }

  if (!attacking) {
    player.vel.x = 0;

    if (kb.pressing("left")) {
      player.vel.x = -1.5;
      player.mirror.x = true;
    }
    else if (kb.pressing("right")) {
      player.vel.x = 1.5;
      player.mirror.x = false;
    }
  }

  player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2);
}
```

ME:

the sound is not playing

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Fox Game</title>
</head>
<body>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.9.0/p5.min.js"></script>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.9.0/addons/p5.sound.min.js"></script>
  <script src="https://p5play.org/v3/p5play.js"></script>
  <script src="sketch.js"></script>
</body>
</html>
```